

MVP Backend Technical Specification

Spring Boot · MySQL · REST API

Version	1.0 — MVP Draft
Date	May 2026
Stack	Java 21 · Spring Boot 3.x · MySQL 8 · JWT
Status	For Engineering Review

1. Introduction & Architecture Overview

This document describes the internals of the MVP backend for a generalized exam-preparation platform. Although the initial deployment targets TAGE MAGE candidates (Objectif 600), the architecture is intentionally exam-agnostic: a single Spring Boot backend can power multiple, independent exam preparation apps — each with its own question bank, categories, and branding — while sharing the same infrastructure.

The backend is built with Spring Boot 3.x on Java 21, stores its primary data in MySQL 8, and exposes a stateless REST API secured with JWT. Role-Based Access Control (RBAC) is enforced via Spring Security throughout every layer.

1.1 Architectural Principles

- Exam-agnostic multi-tenancy: every core entity carries an `exam_type` discriminator so a second app (e.g., GMAT Prep) can reuse the same database and codebase.
- Layered architecture: Controller → Service → Repository (Spring Data JPA) → MySQL.
- Stateless JWT auth: no server-side session state; refresh tokens stored in the DB.
- RBAC via Spring Security: method-level `@PreAuthorize` guards on all sensitive endpoints.
- Analytics-first schema: every user action is persisted at the answer level, enabling rich analytics and a future AI recommendation layer without schema redesign.

1.2 High-Level Component Map

Component	Responsibility
Spring Security + JWT	Authentication, token issuance & validation, role enforcement
User Module	Registration, profile, XP/level, streak, subscription type
Question Bank Module	CRUD for questions, options, categories with hierarchy
Session Module	Training sessions, daily quiz sessions, weekly quiz sessions
Leaderboard Module	Ranked entries by score; global, weekly, per-category
Lesson Module	Rich-text lessons linked to categories and questions
Bookmark Module	User-saved questions for personal revision lists
Subscription Module	Freemium/Pro plans, provider webhooks, access gating
Admin CMS Module	Content management, bulk import, user moderation

2. Database Schema

The schema below is the authoritative MVP definition. All tables use BIGINT surrogate primary keys, TIMESTAMP audit columns, and InnoDB (for foreign key support). The exam_type column present in categories and questions is the generalization hook that makes the backend exam-agnostic.

2.1 users

```
CREATE TABLE users (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  uuid CHAR(36) NOT NULL UNIQUE,  
  username VARCHAR(50) NOT NULL UNIQUE,  
  email VARCHAR(150) NOT NULL UNIQUE,  
  password_hash TEXT NOT NULL,  
  avatar_url TEXT,  
  xp INT DEFAULT 0,  
  level INT DEFAULT 1,  
  streak_days INT DEFAULT 0,  
  subscription_type ENUM('FREE', 'PRO_MONTHLY', 'PRO_ANNUAL') DEFAULT 'FREE',  
  best_category_id BIGINT,  
  is_active TINYINT(1) DEFAULT 1,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  INDEX idx_users_email (email),  
  INDEX idx_users_uuid (uuid)  
);
```

2.2 admins

```
CREATE TABLE admins (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  email VARCHAR(150) NOT NULL UNIQUE,  
  password_hash TEXT NOT NULL,  
  role ENUM('SUPER_ADMIN', 'CONTENT_EDITOR', 'MODERATOR') NOT NULL,  
  is_active TINYINT(1) DEFAULT 1,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

2.3 categories

```
CREATE TABLE categories (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  exam_type VARCHAR(50) NOT NULL DEFAULT 'TAGE_MAGE', -- generalization key  
  name VARCHAR(100) NOT NULL,  
  slug VARCHAR(150) NOT NULL UNIQUE,
```

```

parent_id    BIGINT          REFERENCES categories(id),      -- sub-category hierarchy
description  TEXT,
created_at   TIMESTAMP       DEFAULT CURRENT_TIMESTAMP,
INDEX idx_cat_exam (exam_type)
);

```

2.4 questions

```

CREATE TABLE questions (
  id              BIGINT      AUTO_INCREMENT PRIMARY KEY,
  category_id     BIGINT      NOT NULL REFERENCES categories(id),
  difficulty_level ENUM('EASY','MEDIUM','HARD','EXPERT') NOT NULL,
  question_type   ENUM('MCQ','TRUE_FALSE') DEFAULT 'MCQ',
  question_text   LONGTEXT NOT NULL,
  explanation     LONGTEXT,
  image_url       TEXT,
  is_active       TINYINT(1) DEFAULT 1,
  created_by      BIGINT      REFERENCES admins(id),
  created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  INDEX idx_q_category (category_id),
  INDEX idx_q_difficulty (difficulty_level)
);

```

2.5 question_options

```

CREATE TABLE question_options (
  id              BIGINT      AUTO_INCREMENT PRIMARY KEY,
  question_id     BIGINT      NOT NULL REFERENCES questions(id),
  option_label    VARCHAR(5)  NOT NULL,      -- A, B, C, D, E
  option_text     TEXT        NOT NULL,
  is_correct      TINYINT(1)  DEFAULT 0,
  created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_opt_question (question_id)
);

```

2.6 quiz_sessions

```

CREATE TABLE quiz_sessions (
  id              BIGINT      AUTO_INCREMENT PRIMARY KEY,
  user_id         BIGINT      NOT NULL REFERENCES users(id),
  session_type    ENUM('TRAINING','DAILY','WEEKLY') NOT NULL,
  category_id     BIGINT      REFERENCES categories(id),
  daily_quiz_id   BIGINT      REFERENCES daily_quizzes(id),
  weekly_quiz_id  BIGINT      REFERENCES weekly_quizzes(id),
  started_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  completed_at    TIMESTAMP,

```

```

total_questions  INT          DEFAULT 0,
correct_answers  INT          DEFAULT 0,
score_percentage DECIMAL(5,2),
xp_earned        INT          DEFAULT 0,
duration_seconds INT,
INDEX idx_sess_user (user_id),
INDEX idx_sess_type (session_type)
);

```

2.7 quiz_session_answers

```

CREATE TABLE quiz_session_answers (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  session_id  BIGINT NOT NULL REFERENCES quiz_sessions(id),
  question_id BIGINT NOT NULL REFERENCES questions(id),
  selected_option_id BIGINT REFERENCES question_options(id),
  is_correct  TINYINT(1),
  time_spent_seconds INT,
  answered_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_ans_session (session_id),
  INDEX idx_ans_question (question_id)
);

```

2.8 daily_quizzes & daily_quiz_questions

```

CREATE TABLE daily_quizzes (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  quiz_date   DATE NOT NULL UNIQUE,
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

```

CREATE TABLE daily_quiz_questions (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  daily_quiz_id BIGINT NOT NULL REFERENCES daily_quizzes(id),
  question_id BIGINT NOT NULL REFERENCES questions(id),
  order_index INT NOT NULL
);

```

2.9 weekly_quizzes & weekly_quiz_questions

```

CREATE TABLE weekly_quizzes (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  week_start  DATE NOT NULL,
  week_end    DATE NOT NULL,
  is_active   TINYINT(1) DEFAULT 1,
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

```
CREATE TABLE weekly_quiz_questions (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  weekly_quiz_id BIGINT NOT NULL REFERENCES weekly_quizzes(id),
  question_id  BIGINT NOT NULL REFERENCES questions(id),
  order_index  INT     NOT NULL
);
```

2.10 leaderboard_entries

```
CREATE TABLE leaderboard_entries (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id     BIGINT NOT NULL REFERENCES users(id),
  leaderboard_type ENUM('GLOBAL','WEEKLY','CATEGORY') NOT NULL,
  category_id BIGINT REFERENCES categories(id),
  weekly_quiz_id BIGINT REFERENCES weekly_quizzes(id),
  score       DECIMAL(6,2),
  rank_position INT,
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_lb_user (user_id),
  INDEX idx_lb_type (leaderboard_type)
);
```

2.11 lessons & lesson_questions

```
CREATE TABLE lessons (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  category_id BIGINT NOT NULL REFERENCES categories(id),
  title       VARCHAR(255) NOT NULL,
  content     LONGTEXT NOT NULL,
  is_premium  TINYINT(1) DEFAULT 0,
  created_by  BIGINT REFERENCES admins(id),
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

CREATE TABLE lesson_questions (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  lesson_id   BIGINT NOT NULL REFERENCES lessons(id),
  question_id BIGINT NOT NULL REFERENCES questions(id)
);
```

2.12 bookmarks

```
CREATE TABLE bookmarks (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id     BIGINT NOT NULL REFERENCES users(id),
```

```

question_id BIGINT NOT NULL REFERENCES questions(id),
created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
UNIQUE KEY uq_bookmark (user_id, question_id)
);

```

2.13 subscriptions

```

CREATE TABLE subscriptions (
  id                BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id           BIGINT NOT NULL REFERENCES users(id),
  plan_type         ENUM('PRO_MONTHLY','PRO_ANNUAL') NOT NULL,
  status            ENUM('ACTIVE','CANCELLED','EXPIRED') DEFAULT 'ACTIVE',
  provider          VARCHAR(50),          -- 'stripe', 'apple', 'google'
  provider_subscription_id TEXT,
  start_date        DATETIME NOT NULL,
  end_date          DATETIME NOT NULL,
  created_at        TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_sub_user (user_id)
);

```

2.14 user_question_statistics (MVP — AI Foundation)

Design decision: Included in the MVP from day one. It is the single most valuable table in the system — the foundation for spaced repetition, personalization, and future AI coaching. Building it post-MVP requires costly data backfill.

```

CREATE TABLE user_question_statistics (
  id                BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id           BIGINT NOT NULL REFERENCES users(id),
  question_id       BIGINT NOT NULL REFERENCES questions(id),
  attempts          INT DEFAULT 0,
  correct_attempts  INT DEFAULT 0,
  average_time_sec  DECIMAL(6,2),
  mastery_score     DECIMAL(5,4) DEFAULT 0.0000, -- 0.0 to 1.0
  next_review_at    TIMESTAMP,                  -- SM-2 next due date
  sm2_interval      INT DEFAULT 1,              -- SM-2 interval (days)
  sm2_easiness      DECIMAL(4,2) DEFAULT 2.50,  -- SM-2 easiness factor
  sm2_repetitions   INT DEFAULT 0,
  last_attempted_at TIMESTAMP,
  created_at        TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at        TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  UNIQUE KEY uq_user_question (user_id, question_id),
  INDEX idx_uqs_user   (user_id),
  INDEX idx_uqs_review (user_id, next_review_at),
  INDEX idx_uqs_mastery (user_id, mastery_score)
);

```

foreign keys · SQL

```
-- =====  
  
-- ADD FOREIGN KEY CONSTRAINTS  
  
-- Run these AFTER all tables have been created  
  
-- =====  
  
  
-- users  
ALTER TABLE users  
    ADD CONSTRAINT fk_users_best_category  
        FOREIGN KEY (best_category_id) REFERENCES categories(id)  
        ON DELETE SET NULL;  
  
  
-- subscriptions  
ALTER TABLE subscriptions  
    ADD CONSTRAINT fk_subscriptions_user  
        FOREIGN KEY (user_id) REFERENCES users(id)  
        ON DELETE CASCADE;  
  
  
-- categories (self-referencing for sub-categories)  
ALTER TABLE categories  
    ADD CONSTRAINT fk_categories_parent  
        FOREIGN KEY (parent_id) REFERENCES categories(id)  
        ON DELETE SET NULL;  
  
  
-- questions  
ALTER TABLE questions  
    ADD CONSTRAINT fk_questions_category  
        FOREIGN KEY (category_id) REFERENCES categories(id)  
        ON DELETE RESTRICT;  
  
  
ALTER TABLE questions  
    ADD CONSTRAINT fk_questions_created_by
```

```
FOREIGN KEY (created_by) REFERENCES admins(id)
ON DELETE SET NULL;
```

```
-- question_options
```

```
ALTER TABLE question_options
ADD CONSTRAINT fk_question_options_question
FOREIGN KEY (question_id) REFERENCES questions(id)
ON DELETE CASCADE;
```

```
-- bookmarks
```

```
ALTER TABLE bookmarks
ADD CONSTRAINT fk_bookmarks_user
FOREIGN KEY (user_id) REFERENCES users(id)
ON DELETE CASCADE;
```

```
ALTER TABLE bookmarks
ADD CONSTRAINT fk_bookmarks_question
FOREIGN KEY (question_id) REFERENCES questions(id)
ON DELETE CASCADE;
```

```
-- lessons
```

```
ALTER TABLE lessons
ADD CONSTRAINT fk_lessons_category
FOREIGN KEY (category_id) REFERENCES categories(id)
ON DELETE RESTRICT;
```

```
ALTER TABLE lessons
ADD CONSTRAINT fk_lessons_created_by
FOREIGN KEY (created_by) REFERENCES admins(id)
ON DELETE SET NULL;
```

```
-- lesson_questions
```

```
ALTER TABLE lesson_questions
ADD CONSTRAINT fk_lesson_questions_lesson
```

```
FOREIGN KEY (lesson_id) REFERENCES lessons(id)
ON DELETE CASCADE;
```

```
ALTER TABLE lesson_questions
ADD CONSTRAINT fk_lesson_questions_question
FOREIGN KEY (question_id) REFERENCES questions(id)
ON DELETE CASCADE;
```

```
-- daily_quiz_questions
ALTER TABLE daily_quiz_questions
ADD CONSTRAINT fk_daily_quiz_questions_daily_quiz
FOREIGN KEY (daily_quiz_id) REFERENCES daily_quizzes(id)
ON DELETE CASCADE;
```

```
ALTER TABLE daily_quiz_questions
ADD CONSTRAINT fk_daily_quiz_questions_question
FOREIGN KEY (question_id) REFERENCES questions(id)
ON DELETE RESTRICT;
```

```
-- weekly_quiz_questions
ALTER TABLE weekly_quiz_questions
ADD CONSTRAINT fk_weekly_quiz_questions_weekly_quiz
FOREIGN KEY (weekly_quiz_id) REFERENCES weekly_quizzes(id)
ON DELETE CASCADE;
```

```
ALTER TABLE weekly_quiz_questions
ADD CONSTRAINT fk_weekly_quiz_questions_question
FOREIGN KEY (question_id) REFERENCES questions(id)
ON DELETE RESTRICT;
```

```
-- quiz_sessions
ALTER TABLE quiz_sessions
ADD CONSTRAINT fk_quiz_sessions_user
FOREIGN KEY (user_id) REFERENCES users(id)
```

ON DELETE CASCADE;

ALTER TABLE quiz_sessions

ADD CONSTRAINT fk_quiz_sessions_category
FOREIGN KEY (category_id) REFERENCES categories(id)
ON DELETE SET NULL;

ALTER TABLE quiz_sessions

ADD CONSTRAINT fk_quiz_sessions_daily_quiz
FOREIGN KEY (daily_quiz_id) REFERENCES daily_quizzes(id)
ON DELETE SET NULL;

ALTER TABLE quiz_sessions

ADD CONSTRAINT fk_quiz_sessions_weekly_quiz
FOREIGN KEY (weekly_quiz_id) REFERENCES weekly_quizzes(id)
ON DELETE SET NULL;

-- quiz_session_answers

ALTER TABLE quiz_session_answers

ADD CONSTRAINT fk_quiz_session_answers_session
FOREIGN KEY (session_id) REFERENCES quiz_sessions(id)
ON DELETE CASCADE;

ALTER TABLE quiz_session_answers

ADD CONSTRAINT fk_quiz_session_answers_question
FOREIGN KEY (question_id) REFERENCES questions(id)
ON DELETE RESTRICT;

ALTER TABLE quiz_session_answers

ADD CONSTRAINT fk_quiz_session_answers_selected_option
FOREIGN KEY (selected_option_id) REFERENCES question_options(id)
ON DELETE RESTRICT;

-- leaderboard_entries

```
ALTER TABLE leaderboard_entries
```

```
ADD CONSTRAINT fk_leaderboard_entries_user
```

```
FOREIGN KEY (user_id) REFERENCES users(id)
```

```
ON DELETE CASCADE;
```

```
ALTER TABLE leaderboard_entries
```

```
ADD CONSTRAINT fk_leaderboard_entries_category
```

```
FOREIGN KEY (category_id) REFERENCES categories(id)
```

```
ON DELETE SET NULL;
```

```
ALTER TABLE leaderboard_entries
```

```
ADD CONSTRAINT fk_leaderboard_entries_weekly_quiz
```

```
FOREIGN KEY (weekly_quiz_id) REFERENCES weekly_quizzes(id)
```

```
ON DELETE SET NULL;
```

SM-2 Algorithm Integration (QuestionStatisticsService.java)

After every answered question, `QuestionStatisticsService.updateAfterAnswer()` is called from within `SessionService` to recalculate the three SM-2 fields and the rolling mastery score.

```
public void updateAfterAnswer(Long userId, Long questionId,
                             boolean correct, int timeSpentSec) {
    UserQuestionStatistics s = repo
        .findByUserIdAndQuestionId(userId, questionId)
        .orElseGet(() -> new UserQuestionStatistics(userId, questionId));

    s.setAttempts(s.getAttempts() + 1);
    if (correct) s.setCorrectAttempts(s.getCorrectAttempts() + 1);
    s.setAverageTimeSec(rollingAvg(s, timeSpentSec));

    // SM-2 quality score: correct+fast=5, correct+slow=3, wrong=1
    int quality = computeQuality(correct, timeSpentSec);
    applySmTwo(s, quality);

    // mastery_score = correct_attempts / attempts
    s.setMasteryScore(BigDecimal.valueOf(
        (double) s.getCorrectAttempts() / s.getAttempts()));
    s.setLastAttemptedAt(Instant.now());
    repo.save(s);
}

private void applySmTwo(UserQuestionStatistics s, int q) {
```

```

if (q >= 3) {
    int next = switch (s.getSm2Repetitions()) {
        case 0 -> 1;
        case 1 -> 6;
        default -> Math.round(
            s.getSm2Interval() * s.getSm2Easiness().floatValue());
    };
    s.setSm2Interval(next);
    s.setSm2Repetitions(s.getSm2Repetitions() + 1);
} else {
    s.setSm2Repetitions(0);
    s.setSm2Interval(1);
}
float ef = s.getSm2Easiness().floatValue()
    + (0.1f - (5 - q) * (0.08f + (5 - q) * 0.02f));
s.setSm2Easiness(BigDecimal.valueOf(Math.max(1.3f, ef)));
s.setNextReviewAt(Instant.now()
    .plus(s.getSm2Interval(), ChronoUnit.DAYS));
}

```

AI Personalization — Capabilities Enabled at MVP

Feature	How user_question_statistics powers it
Due-for-review queue	SELECT ... WHERE user_id = ? AND next_review_at <= NOW() ORDER BY next_review_at — surfaces overdue questions for the Revise Mode screen.
Weak topics dashboard	GROUP BY category_id WHERE mastery_score < 0.6 gives per-sub-test weakness rankings displayed on the home screen.
Recommended session	GET /recommendations/session returns the 10 questions with the lowest mastery_score whose next_review_at has passed.
Estimated score projection	Weighted average of mastery_score per category, multiplied by TAGE MAGE sub-test weights, gives a projected score range displayed on the analytics screen.
Adaptive daily quiz	DailyQuizGenerationJob replaces 2 of 10 daily questions with each user's weakest due-for-review items.
Post-MVP: collaborative filtering	Once mastery vectors exist across users, cosine similarity identifies peers with similar weak patterns and cross-recommends effective study paths without changing the schema.

3. Java Class Structure

The project follows a standard layered package structure. Each domain module owns its entity, DTO(s), repository, service, and controller. Cross-cutting concerns (security, exceptions, config) are in dedicated top-level packages.

3.1 Package Layout

```
com.objectif600.backend
├─ config/
│   ├── SecurityConfig.java          (Spring Security, JWT filter chain)
│   ├── JwtConfig.java              (key, expiry settings)
│   └─ CorsConfig.java
├─ security/
│   ├── JwtTokenProvider.java        (generate / validate / parse tokens)
│   ├── JwtAuthenticationFilter.java (OncePerRequestFilter)
│   ├── UserDetailsServiceImpl.java
│   └─ StaffDetailsServiceImpl.java
├─ exception/
│   ├── GlobalExceptionHandler.java (@ControllerAdvice)
│   ├── ResourceNotFoundException.java
│   ├── DuplicateResourceException.java
│   └─ AccessDeniedException.java
├─ user/
│   ├── User.java                   (@Entity)
│   ├── UserRepository.java         (JpaRepository)
│   ├── UserService.java
│   ├── UserController.java
│   └─ dto/ (UserRegistrationDto, UserProfileDto, UserUpdateDto)
├─ staff/
│   ├── Staff.java
│   ├── StaffRepository.java
│   ├── StaffService.java
│   └─ StaffController.java
├─ auth/
│   ├── AuthService.java             (login, register, refresh token)
│   ├── AuthController.java
│   └─ dto/ (LoginRequest, AuthResponse, RefreshRequest)
├─ category/
│   ├── Category.java
│   ├── CategoryRepository.java
│   ├── CategoryService.java
│   └─ CategoryController.java
├─ question/
│   └─ Question.java
```

```
|   ├── QuestionOption.java
|   ├── QuestionRepository.java
|   ├── QuestionService.java
|   ├── QuestionController.java
|   └── dto/
└── session/
    ├── QuizSession.java
    ├── QuizSessionAnswer.java
    ├── QuizSessionRepository.java
    ├── QuizSessionAnswerRepository.java
    ├── SessionService.java
    └── SessionController.java
└── dailyquiz/
    ├── DailyQuiz.java
    ├── DailyQuizQuestion.java
    ├── DailyQuizRepository.java
    ├── DailyQuizService.java
    └── DailyQuizController.java
└── weeklyquiz/
    ├── WeeklyQuiz.java
    ├── WeeklyQuizQuestion.java
    ├── WeeklyQuizService.java
    └── WeeklyQuizController.java
└── leaderboard/
    ├── LeaderboardEntry.java
    ├── LeaderboardRepository.java
    ├── LeaderboardService.java
    └── LeaderboardController.java
└── lesson/
    ├── Lesson.java
    ├── LessonQuestion.java
    ├── LessonService.java
    └── LessonController.java
└── bookmark/
    ├── Bookmark.java
    ├── BookmarkService.java
    └── BookmarkController.java
└── subscription/
    ├── Subscription.java
    ├── SubscriptionService.java
    └── SubscriptionController.java
└── ai/
    ├── UserQuestionStatistics.java
    ├── UserQuestionStatisticsRepository.java
    ├── QuestionStatisticsService.java    (SM-2 algorithm)
    ├── RecommendationService.java        (weak topics + due-review queries)
    └── RecommendationController.java
```

└─ upload/	
└─ CloudinaryConfig.java	(Cloudinary SDK bean)
└─ CloudinaryService.java	(upload, delete, signed URLs)
└─ UploadController.java	(image & file upload endpoints)
└─ importer/	
└─ QuestionCsvImporter.java	(OpenCSV parser + row validator)
└─ CsvQuestionRow.java	(@CsvBindByName mapping DTO)
└─ ImportReportDto.java	(success count, error list)

3.2 Key Entity Classes

User.java

```

@Entity @Table(name = "users")
public class User {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true, nullable = false)
    private String uuid = UUID.randomUUID().toString();

    @Column(unique = true, nullable = false, length = 50)
    private String username;

    @Column(unique = true, nullable = false, length = 150)
    private String email;

    @Column(nullable = false)
    private String passwordHash;

    private String avatarUrl;
    private int xp = 0;
    private int level = 1;
    private int streakDays = 0;

    @Enumerated(EnumType.STRING)
    private SubscriptionType subscriptionType = SubscriptionType.FREE;

    private boolean isActive = true;

    @CreationTimestamp private Instant createdAt;
    @UpdateTimestamp private Instant updatedAt;
}

```

Question.java

```

@Entity @Table(name = "questions")
public class Question {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "category_id", nullable = false)
    private Category category;

    @Enumerated(EnumType.STRING)
    private DifficultyLevel difficultyLevel;

    @Enumerated(EnumType.STRING)
    private QuestionType questionType = QuestionType.MCQ;

    @Column(columnDefinition = "LONGTEXT", nullable = false)
    private String questionText;

    @Column(columnDefinition = "LONGTEXT")
    private String explanation;

    private String imageUrl;
    private boolean isActive = true;

    @OneToMany(mappedBy = "question", cascade = CascadeType.ALL, fetch = FetchType.LAZY)
    private List<QuestionOption> options;

    @CreationTimestamp private Instant createdAt;
    @UpdateTimestamp   private Instant updatedAt;
}

```

QuizSession.java

```

@Entity @Table(name = "quiz_sessions")
public class QuizSession {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    @Enumerated(EnumType.STRING)
    private SessionType sessionType;    // TRAINING | DAILY | WEEKLY

    @ManyToOne(fetch = FetchType.LAZY)

```

```

@JoinColumn(name = "category_id")
private Category category;

@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "daily_quiz_id")
private DailyQuiz dailyQuiz;

@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "weekly_quiz_id")
private WeeklyQuiz weeklyQuiz;

private Instant startedAt;
private Instant completedAt;
private int totalQuestions;
private int correctAnswers;
private BigDecimal scorePercentage;
private int xpEarned;
private int durationSeconds;

@OneToMany(mappedBy = "session", cascade = CascadeType.ALL)
private List<QuizSessionAnswer> answers;
}

```

3.3 Security & JWT Classes

JwtTokenProvider.java (key methods)

```

public class JwtTokenProvider {
    private final String secretKey;
    private final long accessTokenValidityMs = 3_600_000; // 1 h
    private final long refreshTokenValidityMs = 604_800_000; // 7 d

    public String generateAccessToken(UserDetails userDetails) { ... }
    public String generateRefreshToken(String username) { ... }
    public String getUsernameFromToken(String token) { ... }
    public boolean validateToken(String token) { ... }
}

```

SecurityConfig.java (filter chain)

```

@Configuration @EnableWebSecurity @EnableMethodSecurity
public class SecurityConfig {
    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        return http
            .csrf(AbstractHttpConfigurer::disable)
            .sessionManagement(sm -> sm.sessionCreationPolicy(STATELESS))

```

```

        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/v1/auth/**").permitAll()

            .requestMatchers("/api/v1/admin/**").hasAnyRole("SUPER_ADMIN", "CONTENT_EDITOR", "MODERATOR")
                .anyRequest().authenticated())
        .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class)
        .build();
    }
}

```

3.4 Cloudinary Integration (CloudinaryService.java)

All image and file uploads are routed through Cloudinary. The backend never stores binary data in MySQL or on local disk. CloudinaryService wraps the official Cloudinary Java SDK and exposes three operations used across the platform: upload, delete, and URL transformation.

Dependency: implementation 'com.cloudinary:cloudinary-http44:1.36.0' — add to build.gradle

```

// CloudinaryConfig.java
@Configuration
public class CloudinaryConfig {
    @Bean
    public Cloudinary cloudinary(
        @Value("${cloudinary.cloud-name}") String cloudName,
        @Value("${cloudinary.api-key}") String apiKey,
        @Value("${cloudinary.api-secret}") String apiSecret) {
        return new Cloudinary(ObjectUtils.asMap(
            "cloud_name", cloudName,
            "api_key", apiKey,
            "api_secret", apiSecret,
            "secure", true));
    }
}

// CloudinaryService.java
@Service @RequiredArgsConstructor
public class CloudinaryService {

    private final Cloudinary cloudinary;

    /** Upload an image/file from a Spring MultipartFile. */
    public CloudinaryUploadResult upload(MultipartFile file, String folder)
        throws IOException {
        Map<?, ?> result = cloudinary.uploader().upload(
            file.getBytes(),
            ObjectUtils.asMap(

```

```

        "folder",          folder,          // e.g. "questions", "avatars"
        "resource_type",  "auto",
        "use_filename",   true,
        "unique_filename", true));
    return new CloudinaryUploadResult(
        (String) result.get("public_id"),
        (String) result.get("secure_url"),
        (Integer) result.get("bytes"));
}

/** Delete a previously uploaded asset by its Cloudinary public_id. */
public void delete(String publicId) throws IOException {
    cloudinary.uploader().destroy(publicId, ObjectUtils.emptyMap());
}

/** Build a transformation URL (resize, crop, quality auto). */
public String thumbnailUrl(String publicId, int width, int height) {
    return cloudinary.url()
        .transformation(new Transformation()
            .width(width).height(height).crop("fill").quality("auto"))
        .generate(publicId);
}
}

```

Upload use case	Cloudinary folder / notes
Question image	Uploaded to questions/ folder. secure_url stored in questions.image_url.
User avatar	Uploaded to avatars/ folder. secure_url stored in users.avatar_url. Old asset deleted on replace.
Lesson attachment	Uploaded to lessons/ folder. raw resource_type for PDFs; image for illustrations.
Bulk CSV import	CSV is NOT uploaded to Cloudinary. Processed in-memory by QuestionCsvImporter then discarded.

3.5 CSV Bulk Import (QuestionCsvImporter.java)

Admins and content editors can populate the question bank in bulk by uploading a CSV file. The importer is implemented with the OpenCSV library (com.opencsv:opencsv:5.x) and runs synchronously for the MVP; it can be moved to an async queue for very large files post-MVP.

Expected CSV columns: category_slug, difficulty, question_type, question_text, explanation, image_url, option_a, option_b, option_c, option_d, option_e, correct_option

```
// CsvQuestionRow.java - OpenCSV binding DTO
@CsvBindByName annotations map each CSV header to a field.

public class CsvQuestionRow {
    @CsvBindByName(column = "category_slug", required = true) String categorySlug;
    @CsvBindByName(column = "difficulty", required = true) String difficulty;
    @CsvBindByName(column = "question_type") String questionType;
    @CsvBindByName(column = "question_text", required = true) String questionText;
    @CsvBindByName(column = "explanation") String explanation;
    @CsvBindByName(column = "image_url") String imageUrl;
    @CsvBindByName(column = "option_a", required = true) String optionA;
    @CsvBindByName(column = "option_b", required = true) String optionB;
    @CsvBindByName(column = "option_c") String optionC;
    @CsvBindByName(column = "option_d") String optionD;
    @CsvBindByName(column = "option_e") String optionE;
    @CsvBindByName(column = "correct_option", required = true) String correctOption;
}

// QuestionCsvImporter.java - core logic
@Service @RequiredArgsConstructor
public class QuestionCsvImporter {

    private final CategoryRepository categoryRepo;
    private final QuestionRepository questionRepo;

    @Transactional
    public ImportReportDto importFromCsv(MultipartFile file, Long createdByAdminId)
        throws IOException {
        List<CsvQuestionRow> rows = parseCsv(file);
        int success = 0;
        List<String> errors = new ArrayList<>();

        for (int i = 0; i < rows.size(); i++) {
            try {
                validateRow(rows.get(i));
                Question q = mapToEntity(rows.get(i), createdByAdminId);
                questionRepo.save(q);
                success++;
            } catch (Exception ex) {
                errors.add("Row " + (i + 2) + ": " + ex.getMessage());
            }
        }
        return new ImportReportDto(rows.size(), success, errors);
    }

    private List<CsvQuestionRow> parseCsv(MultipartFile file) throws IOException {
```

```
        try (Reader reader = new InputStreamReader(file.getInputStream())) {  
            return new CsvToBeanBuilder<CsvQuestionRow>(reader)  
                .withType(CsvQuestionRow.class)  
                .withIgnoreLeadingWhiteSpace(true)  
                .build().parse();  
        }  
    }  
}  
  
// ImportReportDto – returned to the admin caller  
public record ImportReportDto(  
    int total,  
    int succeeded,  
    List<String> errors) {}
```

4. Main Features

4.1 Authentication & User Management

Feature	Implementation Notes
User registration	BCrypt password hashing; UUID generated on creation; account disabled until activated.
JWT login	AuthService validates credentials, issues access token (1h) + refresh token (7d).
Token refresh	POST /auth/refresh with valid refresh token returns new access token.
Profile management	Users can update username, avatar, and password. Email change requires re-verification.
XP & Level system	XP awarded per session; level = floor(xp / 1000) + 1; computed in UserService.
Daily streak	Incremented by a scheduled job that checks for session completion in the past 24 h.
Account deactivation	Soft-delete via is_active flag; admin can re-enable.
RBAC	ROLE_USER, ROLE_ADMIN, ROLE_SUPER_ADMIN, ROLE_CONTENT_EDITOR, ROLE_MODERATOR via Spring Security.

4.2 Question Bank

Feature	Implementation Notes
Question CRUD	Admin-only via @PreAuthorize. Questions are never hard-deleted; is_active flag used.
Multiple-choice	Up to 5 options per question stored in question_options; exactly one is_correct = 1.
Category hierarchy	parent_id self-join in categories table. exam_type column enables multi-exam support.
Difficulty levels	EASY / MEDIUM / HARD / EXPERT enum. Distribution enforced at daily quiz generation.
Random generation	QuestionService.findRandom(categoryId, difficulty, exclude) uses native query RAND().
Question search	JPA Specification + Pageable; filterable by category, difficulty, exam_type.

Feature	Implementation Notes
Bulk CSV import	Admin endpoint accepts multipart CSV; parsed with OpenCSV; validated before persist.
Explanation	LONGTEXT field; supports Markdown / LaTeX content rendered client-side.

4.3 Training Mode

Feature	Implementation Notes
Unlimited practice	Creates a TRAINING QuizSession; questions streamed one by one or pre-fetched in batch.
Category-specific	Session initialized with categoryId; questions filtered accordingly.
Timed / untimed	Timer is purely client-side; server records time_spent_seconds per answer.
Session summary	On completion, SessionService computes score_percentage, xp_earned, duration_seconds.
XP rewards	+2 XP per correct answer; bonus applied in UserService after session close.
Answer logging	Every answer (correct or not) persisted in quiz_session_answers for analytics.
Resume session	Incomplete session (completed_at IS NULL) can be resumed; answered questions excluded.
Weakest categories	Derived from quiz_session_answers aggregate query, exposed via /analytics/weak-topics.

4.4 Daily Quiz

Feature	Implementation Notes
Auto-generation	Scheduled @Scheduled(cron) job runs nightly; picks 10 questions (40/40/20 split).
One attempt per day	SessionService checks for existing DAILY session with today's daily_quiz_id before creating.
XP & streak reward	+10 XP on completion; streak incremented in UserService.updateStreak().
Daily ranking	LeaderboardService inserts/upserts a WEEKLY entry with today's score after completion.
Pre-generation	AdminService.pregenerateDailyQuizzes(int days) creates N future daily_quizzes in bulk.

4.5 Weekly Quiz (Exam Simulation)

Feature	Implementation Notes
Fixed window	Monday 00:00 UTC → Sunday 23:59 UTC; enforced by WeeklyQuiz.isActive and date range check.
30 questions	6 per sub-test/category; fixed order via order_index.
One attempt per week	SessionService checks for existing WEEKLY session for current weeklyQuizId.
Global countdown	week_end timestamp returned in GET /weekly-quiz/current; client renders countdown.
Leaderboard feed	On session completion, LeaderboardService upserts WEEKLY entry with score_percentage.
+50 XP on submit	Applied regardless of score; UserService.addXp(userId, 50) called from SessionService.

4.6 Leaderboard

Feature	Implementation Notes
Global leaderboard	GLOBAL entries ranked by total XP; re-ranked via scheduled job or on XP change.
Weekly leaderboard	WEEKLY entries for current weeklyQuizId ranked by score_percentage.
Category leaderboard	CATEGORY entries filtered by category_id and leaderboard_type.
User's own rank	LeaderboardRepository.findRankByUserId() uses a MySQL window function subquery.
Pagination	All leaderboard endpoints accept page + size params via Pageable.

4.7 Lessons & Course Library

Feature	Implementation Notes
Rich-text content	LONGTEXT content field supports HTML/Markdown; rendered client-side.
Premium gating	is_premium flag; LessonService checks user.subscriptionType before returning content.
Lesson-question linking	lesson_questions join table; embedded practice questions returned with lesson detail.
Admin CMS	Admin-only CRUD endpoints; publish/unpublish via is_active flag on lesson.

Feature	Implementation Notes
Category-based browsing	GET /lessons?categoryId= returns paginated list sorted by created_at.

4.8 Subscriptions

Feature	Implementation Notes
Plans	PRO_MONTHLY and PRO_ANNUAL stored in subscriptions table.
Providers	Stripe (web), Apple IAP, Google Play; provider + provider_subscription_id stored.
Webhook handling	SubscriptionController exposes /webhooks/stripe, /webhooks/apple, /webhooks/google.
Access gating	SubscriptionService.isProActive(userId) checks subscriptions.end_date > NOW().
Restore purchases	Mobile client sends provider receipt; SubscriptionService validates and re-activates.
Expiry cleanup	@Scheduled job sets status = EXPIRED and user.subscription_type = FREE for expired rows.

5. REST API Endpoints

All endpoints are prefixed with `/api/v1`. Authenticated endpoints require Authorization: Bearer `<access_token>`. Admin endpoints additionally require an admin role claim in the JWT.

5.1 Authentication

Method	Endpoint	Auth	Description
POST	<code>/auth/register</code>	Public	Register new user account
POST	<code>/auth/login</code>	Public	Login; returns access + refresh tokens
POST	<code>/auth/refresh</code>	Public	Exchange refresh token for new access token
POST	<code>/auth/logout</code>	USER	Invalidate refresh token server-side
POST	<code>/auth/admin/login</code>	Public	Admin login; returns admin JWT

5.2 Users

Method	Endpoint	Auth	Description
GET	<code>/users/me</code>	USER	Get authenticated user profile
PUT	<code>/users/me</code>	USER	Update profile (username, avatar)
PUT	<code>/users/me/password</code>	USER	Change password
DELETE	<code>/users/me</code>	USER	Deactivate own account
GET	<code>/users/{id}</code>	ADMIN	Get any user profile (admin)
PUT	<code>/users/{id}/activate</code>	ADMIN	Re-activate disabled user
GET	<code>/users/me/analytics</code>	USER	Personal performance analytics

5.3 Categories

Method	Endpoint	Auth	Description
GET	<code>/categories</code>	USER	List all categories (filterable by exam_type)
GET	<code>/categories/{id}</code>	USER	Get category by ID
POST	<code>/categories</code>	ADMIN	Create category
PUT	<code>/categories/{id}</code>	ADMIN	Update category

Method	Endpoint	Auth	Description
DELETE	/categories/{id}	ADMIN	Soft-delete category

5.4 Questions

Method	Endpoint	Auth	Description
GET	/questions	USER	Paginated question list (filter: category, difficulty)
GET	/questions/{id}	USER	Get question with options
GET	/questions/random	USER	Random question selection for training
POST	/questions	EDITOR	Create question with options
PUT	/questions/{id}	EDITOR	Update question
DELETE	/questions/{id}	ADMIN	Soft-delete question
POST	/questions/import	EDITOR	Bulk import via CSV multipart upload
GET	/questions/search	USER	Full-text search with filters

5.5 Sessions (Training / Daily / Weekly)

Method	Endpoint	Auth	Description
POST	/sessions	USER	Start new session (type in body)
GET	/sessions/{id}	USER	Get session state & answers so far
POST	/sessions/{id}/answers	USER	Submit answer for a question
PUT	/sessions/{id}/complete	USER	Mark session as complete; receive XP summary
GET	/sessions/my	USER	List user's past sessions (paginated)
GET	/sessions/{id}/summary	USER	Detailed session results

5.6 Daily Quiz

Method	Endpoint	Auth	Description
GET	/daily-quiz/today	USER	Get today's daily quiz and questions
GET	/daily-quiz/{date}	USER	Get quiz for a specific past date
POST	/admin/daily-quiz/pregenerate	ADMIN	Pre-generate N future daily quizzes

5.7 Weekly Quiz

Method	Endpoint	Auth	Description
GET	/weekly-quiz/current	USER	Get current active weekly quiz
GET	/weekly-quiz/{id}	USER	Get a specific weekly quiz
GET	/weekly-quiz/history	USER	User's weekly quiz history
POST	/admin/weekly-quiz	ADMIN	Create new weekly quiz
PUT	/admin/weekly-quiz/{id}	ADMIN	Update weekly quiz

5.8 Leaderboard

Method	Endpoint	Auth	Description
GET	/leaderboard/global	USER	Global XP leaderboard (paginated)
GET	/leaderboard/weekly	USER	Current week leaderboard
GET	/leaderboard/category/{id}	USER	Category-specific leaderboard
GET	/leaderboard/me	USER	Authenticated user's rank across boards

5.9 Lessons

Method	Endpoint	Auth	Description
GET	/lessons	USER	List lessons (filter: category, premium)
GET	/lessons/{id}	USER	Lesson detail with embedded questions
POST	/admin/lessons	EDITOR	Create lesson
PUT	/admin/lessons/{id}	EDITOR	Update lesson
DELETE	/admin/lessons/{id}	ADMIN	Delete lesson

5.10 Bookmarks

Method	Endpoint	Auth	Description
GET	/bookmarks	USER	List user's bookmarked questions
POST	/bookmarks/{questionId}	USER	Bookmark a question
DELETE	/bookmarks/{questionId}	USER	Remove bookmark

5.11 Subscriptions

Method	Endpoint	Auth	Description
GET	/subscriptions/me	USER	Get active subscription for user
POST	/subscriptions/activate	USER	Activate subscription (provider + receipt)
POST	/subscriptions/restore	USER	Restore previous subscription
POST	/webhooks/stripe	Public	Stripe webhook handler
POST	/webhooks/apple	Public	Apple IAP webhook handler
POST	/webhooks/google	Public	Google Play webhook handler

5.12 Admin — User Management

Method	Endpoint	Auth	Description
GET	/admin/users	ADMIN	Paginated user list with filters
GET	/admin/users/{id}	ADMIN	User detail with subscription & stats
PUT	/admin/users/{id}/disable	ADMIN	Disable user account
PUT	/admin/users/{id}/activate	ADMIN	Re-enable user account
GET	/admin/analytics/overview	ADMIN	Platform KPIs (DAU, questions, revenue)

5.13 AI Recommendations & Personalization

Method	Endpoint	Auth	Description
GET	/recommendations/session	USER	Returns 10 questions: lowest mastery + due for review today
GET	/recommendations/weak-topics	USER	Top 5 categories by lowest average mastery_score
GET	/recommendations/due-review	USER	Paginated list of questions where next_review_at <= NOW()
GET	/recommendations/score-estimate	USER	Projected TAGE MAGE score range based on mastery vectors
GET	/users/me/statistics	USER	Full user_question_statistics for current user (paginated)

Method	Endpoint	Auth	Description
GET	/users/me/statistics/{questionId}	USER	Per-question SM-2 stats for authenticated user

5.14 File & Image Upload (Cloudinary)

Method	Endpoint	Auth	Description
POST	/upload/avatar	USER	Upload user avatar; returns Cloudinary secure_url
POST	/upload/question-image	EDITOR	Upload image for a question; returns secure_url
POST	/upload/lesson-attachment	EDITOR	Upload PDF or image for a lesson
DELETE	/upload/{publicId}	EDITOR	Delete a Cloudinary asset by its public_id

5.15 CSV Bulk Import

Method	Endpoint	Auth	Description
POST	/admin/questions/import	EDITOR	Upload CSV; returns ImportReportDto {total, succeeded, errors[]}
GET	/admin/questions/import/template	EDITOR	Download blank CSV template with correct headers

6. Cross-Cutting Concerns

6.1 Error Handling

A `@ControllerAdvice` `GlobalExceptionHandler` catches all exceptions and maps them to RFC 7807 Problem Details responses.

Exception	HTTP Status
<code>ResourceNotFoundException</code>	404 Not Found
<code>DuplicateResourceException</code>	409 Conflict
<code>AccessDeniedException</code>	403 Forbidden
<code>MethodArgumentNotValidException</code>	422 Unprocessable Entity
<code>JwtException</code> / <code>ExpiredJwtException</code>	401 Unauthorized
<code>SubscriptionRequiredException</code>	402 Payment Required
Exception (catch-all)	500 Internal Server Error

6.2 Validation

All request DTOs use Jakarta Bean Validation annotations (`@NotBlank`, `@Email`, `@Size`, `@Min`, `@Max`). Controller methods are annotated with `@Valid`.

6.3 Scheduled Jobs

Job	Schedule (cron)
<code>DailyQuizGenerationJob.java</code>	<code>0 0 0 * * ?</code> — midnight UTC daily
<code>StreakResetJob.java</code>	<code>0 5 0 * * ?</code> — 00:05 UTC daily
<code>SubscriptionExpiryJob.java</code>	<code>0 0 1 * * ?</code> — 01:00 UTC daily
<code>LeaderboardRankRefreshJob.java</code>	<code>0 0 * * * ?</code> — hourly
<code>WeeklyQuizCloseJob.java</code>	<code>0 59 23 * * SUN</code> — Sunday 23:59 UTC

6.4 Generalization Strategy (Multi-Exam Support)

To add a second exam (e.g., GMAT Prep) without modifying existing code:

- Insert new rows in categories with `exam_type = 'GMAT'`.

- All question endpoints accept an optional ?examType= query parameter routed through JPA Specification.
- A future exam_apps table can hold app metadata (name, logo, exam_type key) for dynamic multi-tenant branding.
- Spring Security configuration remains unchanged; the same RBAC roles apply across all exam types.
- This design allows deploying the same JAR to serve multiple front-end apps with zero code changes.

6.5 Non-Functional Requirements

Concern	Approach
Security	JWT RS256 signing; BCrypt password hashing (strength 12); HTTPS enforced.
Performance	JPA second-level cache for categories & questions; DB indexes on FK and enum columns.
Scalability	Stateless API; horizontal scaling behind a load balancer; read replicas for analytics queries.
GDPR	User data exportable via GET /users/me/export; DELETE /users/me performs full anonymization.
Observability	Spring Actuator + Micrometer; structured JSON logs via Logback; trace IDs on every request.
Testing	@SpringBootTest integration tests per module; Testcontainers for MySQL; MockMvc for controllers.

7. MVP Build Phases

Phase	Deliverables
Phase 1 — Core	Spring Security + JWT, User module, Category module, Question Bank CRUD, Training sessions, user_question_statistics + SM-2 service
Phase 2 — Content Ops	Cloudinary upload service, CSV bulk import, Lesson CMS, Daily Quiz scheduler, Bookmarks
Phase 3 — Engagement	Weekly Quiz, Leaderboard, AI recommendation endpoints, score projection
Phase 4 — Monetization	Subscription system (Stripe + Apple + Google IAP webhooks), premium content gating

Note: Battle mode, community Q&A, mini-games, push notifications, and gamification badges are post-MVP features requiring additional schema extensions.

8. Deployment

The MVP backend targets a straightforward cloud deployment. The architecture is intentionally simple: one stateless Spring Boot JAR, one managed MySQL instance, and Cloudinary as the external media store. This can be horizontally scaled later with minimal friction.

8.1 Runtime Environment

Concern	Choice & Rationale
JDK	Java 21 LTS (Eclipse Temurin or Amazon Corretto). Required by Spring Boot 3.x; Virtual Threads (Project Loom) available for future throughput gains.
Spring Boot	3.x packaged as an executable fat JAR via ./gradlew bootJar. No application server required.
MySQL	MySQL 8.0+ managed instance (AWS RDS, PlanetScale, or Railway). Enable InnoDB, set character set utf8mb4 for emoji/unicode support in question text.
Cloudinary	Free tier covers the MVP comfortably (25 GB storage, 25 GB bandwidth/month). No server-side storage needed.

8.2 Environment Variables & Secrets

All secrets are injected via environment variables at runtime, never committed to source control. Spring Boot reads them through application.yml property bindings.

```
# application.yml (values come from env vars)
spring:
  datasource:
    url:      ${DB_URL}          # jdbc:mysql://<host>:3306/<db>?useSSL=true
    username: ${DB_USER}
    password: ${DB_PASSWORD}
  jpa:
    hibernate.ddl-auto: validate # NEVER 'create' or 'update' in production
    properties.hibernate.dialect: org.hibernate.dialect.MySQLDialect

jwt:
  secret:          ${JWT_SECRET}          # min 256-bit random string
  access-token-expiry: 3600000             # 1 hour (ms)
  refresh-token-expiry: 604800000         # 7 days (ms)

cloudinary:
  cloud-name: ${CLOUDINARY_CLOUD_NAME}
  api-key:    ${CLOUDINARY_API_KEY}
  api-secret: ${CLOUDINARY_API_SECRET}
```

```

stripe:
  secret-key:      ${STRIPE_SECRET_KEY}
  webhook-secret:  ${STRIPE_WEBHOOK_SECRET}

server:
  port: 8080
  servlet.context-path: /api/v1

```

8.3 Database Migrations (Flyway)

Schema changes are managed with Flyway. The `ddl-auto` is set to `validate` in all environments; Flyway applies versioned migration scripts before the application starts.

```

# Flyway is auto-configured by Spring Boot when the dependency is present.
# Migration scripts live at: src/main/resources/db/migration/

# Naming convention:
# V1__create_users.sql
# V2__create_categories_and_questions.sql
# V3__create_quiz_sessions.sql
# V4__create_user_question_statistics.sql
# V5__create_leaderboard_entries.sql
# ...

# application.yml addition:
spring:
  flyway:
    enabled: true
    locations: classpath:db/migration
    baseline-on-migrate: false

```

8.4 Containerization (Docker)

The application is packaged as a Docker image using a multi-stage build to keep the final image lean.

```

# Dockerfile
FROM eclipse-temurin:21-jdk-alpine AS builder
WORKDIR /app
COPY . .
RUN ./gradlew bootJar --no-daemon

FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=builder /app/build/libs/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]

```

```
# docker-compose.yml (local development)
services:
  api:
    build: .
    ports: ["8080:8080"]
    environment:
      DB_URL:      jdbc:mysql://db:3306/objectif600
      DB_USER:     root
      DB_PASSWORD: secret
    depends_on: [db]
  db:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: secret
      MYSQL_DATABASE:      objectif600
    volumes: ["mysql_data:/var/lib/mysql"]
volumes:
  mysql_data:
```

8.5 Recommended Hosting Options

Platform	Notes
Railway (recommended for MVP)	One-click MySQL + Spring Boot deployment. Auto-deploys on git push. Managed secrets. Free tier sufficient for early users. Scales vertically with a slider.
Render	Similar to Railway. Dockerfile-based deploy. Managed PostgreSQL also available if migrating away from MySQL.
AWS (ECS + RDS)	Production-grade target. Run the Docker image on ECS Fargate; RDS MySQL for the database; ALB for load balancing. More setup but full control and SLA.
DigitalOcean App Platform	App Platform detects the Dockerfile automatically. Managed MySQL add-on available. Good price/performance for early production.
Fly.io	Global edge deployment. Low latency for distributed mobile users. fly.toml-based config; persistent volumes for Flyway state.

8.6 CI/CD Pipeline

A GitHub Actions workflow handles test, build, and deploy on every push to main.

```
# .github/workflows/deploy.yml
name: Build & Deploy
on:
  push:
```

```

branches: [main]

jobs:
  test-and-build:
    runs-on: ubuntu-latest
    services:
      mysql:
        image: mysql:8.0
        env: { MYSQL_ROOT_PASSWORD: test, MYSQL_DATABASE: testdb }
        ports: ['3306:3306']
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { java-version: '21', distribution: 'temurin' }
      - run: ./gradlew test bootJar --no-daemon
      - uses: docker/build-push-action@v5
        with:
          push: true
          tags: registry.example.com/objectif600-api:${{ github.sha }}

  deploy:
    needs: test-and-build
    runs-on: ubuntu-latest
    steps:
      - name: Deploy to Railway / Render / ECS
        run: |
          # Platform-specific deploy command here
          # e.g.: railway up OR render deploy OR aws ecs update-service ...

```

8.7 Health & Observability

Concern	Implementation
Health check	Spring Actuator exposes GET /actuator/health. Used by load balancer health probes and Docker HEALTHCHECK.
Metrics	Micrometer + Actuator exports JVM, HTTP, and custom business metrics (sessions started, XP awarded). Scraped by Prometheus or sent to Datadog.
Structured logs	Logback configured for JSON output in production. Each log line includes traceId, userId, and endpoint for easy filtering in CloudWatch / Papertrail / Loki.
Alerting	Alert on p99 API latency > 500ms, error rate > 1%, and DB connection pool exhaustion.
Backups	Automated daily MySQL snapshots via the managed DB provider. Retain 7 days minimum.

Production checklist: Before go-live: enable SSL on all endpoints, rotate all secrets from defaults, set Flyway baseline-on-migrate false, configure DB connection pool (HikariCP max-pool-size=10 for MVP), enable Spring Actuator with management.endpoints.web.exposure.include=health,metrics only.

LearnCore

MVP Backend Technical Specification

Spring Boot · MySQL · REST API · Multi-Tenant Architecture

Version	3.0 — Staff Hierarchy Draft
Date	June 2026
Stack	Java 21 · Spring Boot 3.x · MySQL 8 · JWT
Status	For Engineering Review
Key Change v2	Full multi-tenant architecture (tenant_id on all content tables)
Key Change v3	Two-tier staff hierarchy: Platform staff (Learncore) vs Tenant staff (Partner)

1. Introduction & Architecture Overview

LearnCore is a multi-tenant, exam-agnostic e-learning backend owned and operated by Dodibo. The platform supports two completely independent staff hierarchies sharing a single staff table: Learncore platform staff (Dodibo's own team) and tenant staff (each partner company's own team). End-users (learners) are always scoped to exactly one tenant.

1.1 The Two Staff Worlds

✦ **NEW (v3.0) — This is the core hierarchy constraint of v3.0. Platform staff and tenant staff are discriminated by `tenant_id = NULL` vs `tenant_id = <X>`.**

World	tenant_id	Who they are	What they can do
Learncore Platform Staff	NULL	Dodibo's own team (you, Tom, Lucy, ...)	Manage tenants, view cross-tenant analytics, configure the platform
Tenant Staff	= tenant's ID	Partner company employees (Brad, William, Ali, ...)	Manage their own tenant's content, users, and staff only

1.2 Concrete Role Examples

Person	Role	tenant_id	Can do
You (owner)	SUPER_ADMIN	NULL	Everything — God mode
Tom	PLATFORM_MANAGER	NULL	Create/update/disable tenants, provision TENANT_ADMIN accounts
Lucy	PLATFORM_ANALYST	NULL	Read-only cross-tenant reports and KPIs — no write access
Brad	TENANT_ADMIN	X	Full admin of Tenant X: manage X's staff, content, and learners
William	COURSE_EDITOR	X	Create/edit lessons and courses within Tenant X only

Ali	QUESTION_EDITOR	X	Create/edit questions and categories within Tenant X only
-----	-----------------	---	---

1.3 Architectural Principles

- Two-tier staff hierarchy: platform staff (tenant_id = NULL) vs tenant staff (tenant_id = <X>). A single staff table with nullable tenant_id is the discriminator.
- Tenant isolation by design: every content entity carries tenant_id. All repository queries are automatically scoped via TenantAwareRepository.
- TENANT_ADMIN can create/manage staff within their own tenant — but cannot elevate anyone to a platform-level role.
- Platform staff cannot be created by any tenant. Only SUPER_ADMIN or PLATFORM_MANAGER can create platform-level staff accounts.
- Stateless JWT auth: tenant_id, staff world (PLATFORM or TENANT), and role are encoded in JWT claims.
- Layered architecture: Controller → Service → Repository (Spring Data JPA) → MySQL.

1.4 High-Level Component Map

Component	Responsibility
Spring Security + JWT	Authentication, token issuance, role enforcement, tenant claim injection
Tenant Module	Tenant CRUD — accessible to SUPER_ADMIN and PLATFORM_MANAGER
Platform Staff Module	Learncore team management — SUPER_ADMIN only
Tenant Staff Module	Partner team management — TENANT_ADMIN (own tenant) + PLATFORM_MANAGER
User Module	Learner registration, profile, XP/level, streak, subscription — scoped per tenant
Question Bank Module	CRUD for questions, options, categories — QUESTION_EDITOR + TENANT_ADMIN
Session Module	Training/daily/weekly sessions — scoped per tenant
Leaderboard Module	Rankings — scoped per tenant

Lesson / Course Module	Rich-text lessons — COURSE_EDITOR + TENANT_ADMIN
Bookmark Module	User-saved questions
Subscription Module	Freemium/Pro plans, provider webhooks
Admin CMS Module	Content management, bulk import, user moderation — per tenant

2. Database Schema

2.0 tenants

```
CREATE TABLE tenants (  
  id          BIGINT          AUTO_INCREMENT PRIMARY KEY,  
  uuid        CHAR(36)        NOT NULL UNIQUE,    -- prefix: tn_  
  slug        VARCHAR(80)     NOT NULL UNIQUE,    -- e.g. 'objectif600', 'gmat-pro'  
  name        VARCHAR(150)    NOT NULL,  
  exam_focus  VARCHAR(50),  
  logo_url    TEXT,  
  is_active   TINYINT(1)      DEFAULT 1,  
  created_at  TIMESTAMP       DEFAULT CURRENT_TIMESTAMP,  
  updated_at  TIMESTAMP       DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
);
```

2.1 staff — two-tier hierarchy

✦ **NEW (v3.0) — tenant_id = NULL → Learncore platform staff. tenant_id = X → Tenant X's own staff. These are two separate worlds in the same table.**

```
CREATE TABLE staff (  
  id          BIGINT          AUTO_INCREMENT PRIMARY KEY,  
  uuid        CHAR(36)        NOT NULL UNIQUE,    -- prefix: st_  
  tenant_id   BIGINT          REFERENCES tenants(id) ON DELETE CASCADE,  
  -- NULL    = Learncore platform staff (Dodibo team)  
  -- NOT NULL = Tenant's own staff (partner employees)  
  firstname   VARCHAR(100)    NOT NULL,  
  lastname    VARCHAR(100)    NOT NULL,  
  email       VARCHAR(150)    NOT NULL,  
  password_hash TEXT          NOT NULL,  
  position_name VARCHAR(100),      -- free-text job title  
  is_active   TINYINT(1)      DEFAULT 1,  
  created_by  BIGINT          REFERENCES staff(id) ON DELETE SET NULL,  
  created_at  TIMESTAMP       DEFAULT CURRENT_TIMESTAMP,  
  updated_at  TIMESTAMP       DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
```

```

-- email must be unique within the same world (platform or per-tenant)
UNIQUE KEY uq_staff_email_per_scope (tenant_id, email),
INDEX idx_staff_tenant (tenant_id)
);

```

2.2 staff_roles

✦ **NEW (v3.0) — Platform roles (SUPER_ADMIN, PLATFORM_MANAGER, PLATFORM_ANALYST) are only assignable when tenant_id IS NULL. Tenant roles are only assignable when tenant_id IS NOT NULL. This constraint is enforced at the service layer.**

```

CREATE TABLE staff_roles (
  staff_id BIGINT NOT NULL REFERENCES staff(id) ON DELETE CASCADE,
  role_name ENUM(
    -- Platform-level roles (tenant_id = NULL only)
    'SUPER_ADMIN',
    'PLATFORM_MANAGER',
    'PLATFORM_ANALYST',
    -- Tenant-level roles (tenant_id NOT NULL only)
    'TENANT_ADMIN',
    'COURSE_EDITOR',
    'QUESTION_EDITOR'
  ) NOT NULL,
  PRIMARY KEY (staff_id, role_name)
);

```

⚠ **Service-layer guard: StaffService.assignRole() rejects any attempt to assign a platform-level role to a tenant-scoped staff member, or a tenant-level role to a platform staff member.**

2.3 users

```

CREATE TABLE users (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  uuid CHAR(36) NOT NULL UNIQUE, -- prefix: us_
  tenant_id BIGINT NOT NULL REFERENCES tenants(id),

```

```

    firstname          VARCHAR(100),
    lastname           VARCHAR(100),
    username            VARCHAR(50)   NOT NULL,
    email               VARCHAR(150)  NOT NULL,
    password_hash       TEXT          NOT NULL,
    avatar_url          TEXT,
    xp                  INT            DEFAULT 0,
    level               INT            DEFAULT 1,
    streak_days         INT            DEFAULT 0,
    subscription_type   ENUM('FREE','PRO_MONTHLY','PRO_ANNUAL') DEFAULT 'FREE',
    best_category_id    BIGINT,
    is_active           TINYINT(1)    DEFAULT 1,
    created_at          TIMESTAMP      DEFAULT CURRENT_TIMESTAMP,
    updated_at          TIMESTAMP      DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    UNIQUE KEY uq_user_email_per_tenant    (tenant_id, email),
    UNIQUE KEY uq_user_username_per_tenant (tenant_id, username),
    INDEX idx_users_tenant (tenant_id),
    INDEX idx_users_uuid    (uuid)
);

```

2.4 – 2.12 Content tables

All content tables (categories, questions, question_options, quiz_sessions, quiz_session_answers, daily_quizzes, weekly_quizzes, leaderboard_entries, lessons, lesson_questions, bookmarks, subscriptions, user_question_statistics) are unchanged from v2.0. Each carries tenant_id NOT NULL (except child tables that inherit tenancy from their parent).

The created_by column on questions and lessons now references staff(id) instead of the deprecated admins table.

3. Role & Permission Matrix

3.1 Full Permission Matrix

Action	SUPER_ADM IN	PLATFORM_M GR	PLATFORM_ANAL YST	TENANT_AD MIN	COURSE_EDIT OR	QUESTION_EDIT OR
Create / delete tenants	☒	☒	☐	☐	☐	☐
Update tenant settings	☒	☒	☐	☐	☐	☐
Disable / re-enable tenant	☒	☒	☐	☐	☐	☐
View cross-tenant analytics	☒	☐	☒	☐	☐	☐
Create platform staff (Tom, Lucy...)	☒	☒	☐	☐	☐	☐
Assign platform-level roles	☒	☐	☐	☐	☐	☐
Create tenant staff (Brad, William...)	☒	☒	☐	☒	☐	☐
Assign tenant-level roles	☒	☒	☐	☒	☐	☐

3.2 Key Constraints

- A TENANT_ADMIN can only create staff with tenant-level roles (TENANT_ADMIN, COURSE_EDITOR, QUESTION_EDITOR). Attempting to assign SUPER_ADMIN, PLATFORM_MANAGER, or PLATFORM_ANALYST throws a 403.
- A TENANT_ADMIN can only create staff within their own tenant. The service layer hardcodes the new staff member's tenant_id from TenantContext — the caller cannot override it.
- PLATFORM_MANAGER (Tom) can create a TENANT_ADMIN for any tenant (to bootstrap a new partner) but cannot assign platform-level roles — only SUPER_ADMIN can do that.
- PLATFORM_ANALYST (Lucy) is strictly read-only. No write endpoints are accessible. Enforced via @PreAuthorize on all mutating service methods.
- A staff member can only hold roles from one world. The system rejects a request to assign both PLATFORM_MANAGER and TENANT_ADMIN to the same staff record.

4. Java Class Structure

4.1 Package Layout

✦ **NEW (v3.0)** — `staff/` is now split into `platform-staff/` and `tenant-staff/` sub-concerns within the same module. `common/tenant/` provides the isolation infrastructure.

```
com.dodibo.learncore
├─ config/
│   ├── SecurityConfig.java
│   ├── JwtConfig.java
│   └─ CorsConfig.java
├─ common/
│   ├── BaseEntity.java
│   ├── UuidGenerator.java
│   └─ tenant/
│       ├── TenantContext.java          (ThreadLocal<Long> tenantId)
│       ├── TenantAwareRepository.java  (auto-scopes queries to tenantId)
│       └─ TenantIsolationAspect.java   (AOP guard on all @Service writes)
├─ security/
│   ├── JwtTokenProvider.java           (encodes tenantId + staffWorld + role)
│   ├── JwtAuthenticationFilter.java    (populates TenantContext from JWT)
│   ├── UserDetailsServiceImpl.java     (loads User by email + tenantId)
│   └─ StaffDetailsServiceImpl.java     (loads Staff; handles both worlds)
├─ exception/
│   ├── GlobalExceptionHandler.java
│   ├── ResourceNotFoundException.java
│   ├── DuplicateResourceException.java
│   ├── TenantAccessDeniedException.java
│   ├── RoleAssignmentException.java    [NEW - wrong-world role assignment]
│   └─ AccessDeniedException.java
├─ tenant/
│   ├── Tenant.java                    (@Entity)
│   ├── TenantRepository.java
│   ├── TenantService.java
│   ├── TenantController.java          (SUPER_ADMIN + PLATFORM_MANAGER)
│   └─ dto/
```

```

├─ staff/
|   ├─ Staff.java                (@Entity - extends User, tenant_id nullable)
|   ├─ StaffRole.java           (@Entity - staff_roles join table)
|   ├─ StaffRepository.java
|   ├─ StaffService.java        (role-world validation logic lives here)
|   ├─ StaffController.java
|   └─ dto/
|       ├─ CreatePlatformStaffDto.java  (SUPER_ADMIN / PLATFORM_MANAGER use)
|       └─ CreateTenantStaffDto.java    (TENANT_ADMIN / PLATFORM_MANAGER use)
├─ auth/
|   ├─ AuthService.java
|   ├─ AuthController.java
|   └─ dto/
|       ├─ UserLoginRequest.java
|       ├─ StaffLoginRequest.java
|       ├─ SuperAdminLoginRequest.java
|       └─ AuthResponse.java
├─ user/
├─ category/
├─ question/
├─ session/
├─ dailyquiz/
├─ weeklyquiz/
├─ leaderboard/
├─ lesson/
├─ bookmark/
├─ subscription/
├─ ai/
├─ upload/
└─ importer/

```

4.2 Key Classes

Staff.java

```

@Entity @Table(name = "staff")

@Getter @Setter @SuperBuilder @AllArgsConstructor @NoArgsConstructor

```

```

@EntityListeners(AuditingEntityListener.class)

public class Staff extends User {    // inherits id, uuid, email, passwordHash etc.


    // NULL   = Learncore platform staff
    // NOT NULL = tenant's own staff

    @Column(name = "tenant_id")    // nullable
    private Long tenantId;


    private String firstname;
    private String lastname;
    private String positionName;


    @OneToMany(mappedBy = "staff", cascade = CascadeType.ALL, fetch = FetchType.EAGER)
    private Set<StaffRole> roles = new HashSet<>();


    public boolean isPlatformStaff() { return tenantId == null; }
    public boolean isTenantStaff()    { return tenantId != null; }


    @Override
    protected String uidPrefix() { return "st"; }
}

```

StaffRole.java

```

@Entity @Table(name = "staff_roles")
@Getter @Setter @NoArgsConstructor
public class StaffRole {


    @EmbeddedId
    private StaffRoleId id;


    @ManyToOne(fetch = FetchType.LAZY)
    @MapsId("staffId")
    private Staff staff;


    @Enumerated(EnumType.STRING)
    @Column(name = "role_name", nullable = false)

```

```

private RoleName roleName;

public enum RoleName {
    // Platform-level (tenant_id = NULL only)
    SUPER_ADMIN, PLATFORM_MANAGER, PLATFORM_ANALYST,
    // Tenant-level (tenant_id NOT NULL only)
    TENANT_ADMIN, COURSE_EDITOR, QUESTION_EDITOR
}

public boolean isPlatformRole() {
    return roleName == RoleName.SUPER_ADMIN
        || roleName == RoleName.PLATFORM_MANAGER
        || roleName == RoleName.PLATFORM_ANALYST;
}
}

```

StaffService.java — role-world validation

```

@Service @RequiredArgsConstructor
public class StaffService {

    public Staff createTenantStaff(CreateTenantStaffDto dto, Staff creator) {
        // Only TENANT_ADMIN or platform staff can create tenant staff
        Long targetTenantId = creator.isPlatformStaff()
            ? dto.getTenantId() // PLATFORM_MANAGER specifies which
tenant
            : creator.getTenantId(); // TENANT_ADMIN: always own tenant

        // Guard: tenant staff cannot be assigned platform-level roles
        dto.getRoles().forEach(role -> {
            if (new StaffRole(role).isPlatformRole())
                throw new RoleAssignmentException(
                    "Cannot assign platform role '" + role + "' to tenant staff.");
        });

        Staff newStaff = Staff.builder()
            .tenantId(targetTenantId) // hardcoded — caller cannot override

```

```

        .email(dto.getEmail())
        .firstname(dto.getFirstname())
        .lastname(dto.getLastname())
        .positionName(dto.getPositionName())
        .passwordHash(passwordEncoder.encode(dto.getPassword()))
        .build();

    return staffRepository.save(newStaff);
}

public Staff createPlatformStaff(CreatePlatformStaffDto dto) {
    // tenantId is always NULL for platform staff
    dto.getRoles().forEach(role -> {
        if (!new StaffRole(role).isPlatformRole())
            throw new RoleAssignmentException(
                "Cannot assign tenant role '" + role + "' to platform staff.");
    });
    // ... build and save ...
}
}

```

AuthService.java — three login flows

```

// Flow 1: Learner login
// POST /auth/login + X-Tenant-Slug header
// → resolves tenant from slug, validates user credentials
// → JWT includes: sub=email, tenantId=X, world=USER, role=ROLE_USER

// Flow 2: Staff login (both platform and tenant staff)
// POST /auth/staff/login + optional X-Tenant-Slug header
// → if X-Tenant-Slug present: look up tenant staff by (email, tenantId)
// → if X-Tenant-Slug absent: look up platform staff by email (tenantId=NULL)
// → JWT includes: sub=email, tenantId=X|null, world=STAFF, roles=[...]

// Flow 3: Super admin shortcut (no header needed)
// POST /auth/superadmin/login
// → validates SUPER_ADMIN credentials directly

```

```
// → JWT includes: sub=email, tenantId=null, world=STAFF, role=SUPER_ADMIN

// TenantContext behaviour:
// Platform staff → TenantContext.getTenantId() == null (no scope restriction)
// Tenant staff   → TenantContext.getTenantId() == X   (scoped to tenant X)
// Learner        → TenantContext.getTenantId() == X   (scoped to tenant X)
```

5. REST API Endpoints

5.0 Platform Management (Learncore internal)

Method	Endpoint	Who	Description
GET	/platform/tenants	SUPER_ADMIN, PLATFORM_MANAGER	List all tenants
POST	/platform/tenants	SUPER_ADMIN, PLATFORM_MANAGER	Onboard new partner tenant
GET	/platform/tenants/{id}	SUPER_ADMIN, PLATFORM_MANAGER	Tenant detail
PUT	/platform/tenants/{id}	SUPER_ADMIN, PLATFORM_MANAGER	Update tenant settings
PUT	/platform/tenants/{id}/disable	SUPER_ADMIN, PLATFORM_MANAGER	Deactivate tenant
GET	/platform/analytics	SUPER_ADMIN, PLATFORM_ANALYST	Cross-tenant KPIs (DAU, questions, revenue)
GET	/platform/staff	SUPER_ADMIN, PLATFORM_MANAGER	List Learncore platform staff
POST	/platform/staff	SUPER_ADMIN, PLATFORM_MANAGER	Create platform staff account (Tom, Lucy...)
DELETE	/platform/staff/{id}	SUPER_ADMIN	Remove platform staff member
PUT	/platform/tenants/{id}/staff	SUPER_ADMIN, PLATFORM_MANAGER	Provision first TENANT_ADMIN for a new partner

5.1 Authentication

Method	Endpoint	Who	Description
POST	/auth/register	Public	Learner registration. Requires X-Tenant-Slug header.

POST	/auth/login	Public	Learner login. Requires X-Tenant-Slug header.
POST	/auth/staff/login	Public	Staff login (platform or tenant). X-Tenant-Slug optional: present = tenant staff, absent = platform staff.
POST	/auth/superadmin/login	Public	SUPER_ADMIN shortcut — no X-Tenant-Slug needed.
POST	/auth/refresh	Public	Exchange refresh token for new access token.
POST	/auth/logout	Any	Invalidate refresh token.

5.2 Tenant Staff Management (per-tenant)

Method	Endpoint	Who	Description
GET	/admin/staff	TENANT_ADMIN, PLATFORM_MANAGER	List staff for this tenant
POST	/admin/staff	TENANT_ADMIN, PLATFORM_MANAGER	Create new staff member for this tenant (William, Ali...)
GET	/admin/staff/{id}	TENANT_ADMIN, PLATFORM_MANAGER	Get staff detail
PUT	/admin/staff/{id}	TENANT_ADMIN	Update staff info / role within own tenant
PUT	/admin/staff/{id}/disable	TENANT_ADMIN	Disable staff account (own tenant only)
DELETE	/admin/staff/{id}	TENANT_ADMIN	Remove staff from tenant

5.3 All other endpoints — unchanged from v2.0

Users, Categories, Questions, Sessions, Daily Quiz, Weekly Quiz, Leaderboard, Lessons, Bookmarks, Subscriptions, AI Recommendations, File Upload, and CSV Import endpoints retain the same structure as v2.0. The following rules apply uniformly:

- All service methods read `TenantContext.getTenantId()` and scope every repository call to that tenant.
- Platform staff (`tenantId = null`) bypass tenant scoping and can access any tenant's data when calling admin endpoints.
- `TENANT_ADMIN`, `COURSE_EDITOR`, and `QUESTION_EDITOR` are always scoped to their own tenant — the service layer enforces this regardless of what IDs are passed in the request.
- `PLATFORM_ANALYST` can call read-only endpoints across tenants via the `/platform/analytics` namespace only.

6. Security Configuration

6.1 SecurityConfig.java

```
@Configuration @EnableWebSecurity @EnableMethodSecurity
public class SecurityConfig {

    @Bean

    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {

        return http

            .csrf(AbstractHttpConfigurer::disable)

            .sessionManagement(sm -> sm.sessionCreationPolicy(STATELESS))

            .authorizeHttpRequests(auth -> auth

                .requestMatchers("/api/v1/auth/**").permitAll()

                // Platform namespace – Learncore team only

                .requestMatchers("/api/v1/platform/**")

                    .hasAnyRole("SUPER_ADMIN", "PLATFORM_MANAGER", "PLATFORM_ANALYST")

                // Admin namespace – tenant staff only

                .requestMatchers("/api/v1/admin/**")

                    .hasAnyRole("TENANT_ADMIN", "COURSE_EDITOR", "QUESTION_EDITOR")

                .anyRequest().authenticated())

            .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class)

            .build();

    }

}
```

6.2 Method-level guards (examples)

```
// TenantStaffController.java

@PostMapping("/admin/staff")

@PreAuthorize("hasRole('TENANT_ADMIN') or hasRole('PLATFORM_MANAGER')")

public ResponseEntity<StaffDto> createTenantStaff(@RequestBody @Valid
CreateTenantStaffDto dto) { ... }


// QuestionController.java

@PostMapping("/admin/questions")

@PreAuthorize("hasAnyRole('QUESTION_EDITOR', 'TENANT_ADMIN', 'SUPER_ADMIN')")
```

```
public ResponseEntity<QuestionDto> createQuestion(@RequestBody @Valid QuestionDto dto) {  
    ... }  
}
```

```
// LessonController.java
```

```
@PostMapping("/admin/lessons")
```

```
@PreAuthorize("hasAnyRole('COURSE_EDITOR','TENANT_ADMIN','SUPER_ADMIN')")
```

```
public ResponseEntity<LessonDto> createLesson(@RequestBody @Valid LessonDto dto) { ... }
```

```
// AnalyticsController.java
```

```
@GetMapping("/platform/analytics")
```

```
@PreAuthorize("hasAnyRole('SUPER_ADMIN','PLATFORM_ANALYST')")
```

```
public ResponseEntity<PlatformAnalyticsDto> getCrossTenantAnalytics() { ... }
```

7. Cross-Cutting Concerns

7.1 Tenant Isolation — Defence in Depth

Layer	Mechanism
JWT claim	tenant_id + staffWorld encoded in every token. Platform staff get tenantId=null; tenant staff get tenantId=X.
TenantContext	ThreadLocal populated by JwtAuthenticationFilter on every request. Cleared in finally block after filter chain.
TenantAwareRepository	Appends WHERE tenant_id = ? to all queries. When tenantId=null (platform staff), the filter is skipped — cross-tenant access allowed.
TenantIsolationAspect	AOP @Around on all @Service write methods. Asserts entity.tenantId == TenantContext.getTenantId() before save/delete. Skipped for platform staff.
Role-world guard	StaffService.assignRole() rejects cross-world role assignment (platform role to tenant staff or vice versa). Throws RoleAssignmentException (400).
DB UNIQUE constraints	Unique keys scoped per tenant prevent accidental data bleed between tenants at the DB level.

7.2 Error Handling

Exception	HTTP Status
ResourceNotFoundException	404 Not Found
DuplicateResourceException	409 Conflict
TenantAccessDeniedException	403 Forbidden — resource does not belong to your tenant
RoleAssignmentException [NEW]	400 Bad Request — cross-world role assignment attempted
AccessDeniedException	403 Forbidden
MethodArgumentNotValidException	422 Unprocessable Entity
JwtException / ExpiredJwtException	401 Unauthorized
SubscriptionRequiredException	402 Payment Required

Exception (catch-all)	500 Internal Server Error
-----------------------	---------------------------

7.3 Scheduled Jobs

Job	Behaviour
DailyQuizGenerationJob	Iterates all active tenants; generates one daily quiz per tenant per night.
StreakResetJob	Iterates active tenants; resets streaks per tenant.
SubscriptionExpiryJob	Iterates active tenants; expires subscriptions and resets user.subscriptionType.
LeaderboardRankRefreshJob	Refreshes leaderboard rankings per tenant independently.
WeeklyQuizCloseJob	Closes each tenant's active weekly quiz on Sunday 23:59 UTC.

8. MVP Build Phases

Phase	Deliverables
Phase 0 — Multi-Tenancy + Staff Hierarchy Foundation	tenants table. Staff entity with nullable tenantId. StaffRole with role-world validation. TenantContext, TenantAwareRepository, TenantIsolationAspect. Three JWT auth flows. SUPER_ADMIN + PLATFORM_MANAGER + PLATFORM_ANALYST bootstrap. /platform/** namespace.
Phase 1 — Core	User module (tenant-scoped). Tenant staff CRUD (TENANT_ADMIN self-service). Category + Question Bank CRUD. Training sessions. user_question_statistics + SM-2.
Phase 2 — Content Ops	Cloudinary upload. CSV bulk import. Lesson / Course CMS (COURSE_EDITOR). Daily Quiz scheduler per tenant. Bookmarks.
Phase 3 — Engagement	Weekly Quiz. Leaderboard (per-tenant). AI recommendations. Score projection.
Phase 4 — Monetization	Subscription system (Stripe + Apple + Google). Premium content gating.

9. Deployment

Unchanged from v2.0 — single Spring Boot JAR, single MySQL database, Cloudinary for media. Refer to section 8 of v2.0 for Docker setup, Flyway migrations, CI/CD, and hosting options.

9.1 Flyway Migration Order

```
V1__create_tenants.sql
V2__create_staff_and_staff_roles.sql    -- tenant_id nullable; role_name enum includes
all 6 roles
V3__create_users.sql
V4__create_categories_and_questions.sql
V5__create_quiz_sessions.sql
V6__create_user_question_statistics.sql
V7__create_leaderboard_entries.sql
V8__create_lessons_and_bookmarks.sql
V9__create_subscriptions.sql
V10__seed_first_tenant.sql              -- seeds Objectif 600
V11__seed_superadmin.sql                -- seeds Dodibo SUPER_ADMIN account
V12__seed_platform_staff_roles.sql      -- seeds PLATFORM_MANAGER, PLATFORM_ANALYST role
definitions
```